# CS263 Project Report

Joshua You

Link to GitHub

## Motivation

The goal of this project was to investigate a basic formalization of a stackless, asymmetric coroutine splitting transform inspired by the coroutine system from the LuisaCompute project [1]. This transform takes an ordinary program annotated with special `suspend` markers and converts it into a set of cooperatively-scheduled coroutines. In spirit, this is similar to the async functionality in many programming languages, but is more low-level. It would be possible to build generators using this transform, as well as async / await (each `.await` is a suspension point).

There are two main components to this:

1. **Control flow splitting**: take the CFG of the program and perform reachability analysis from special annotations (`suspend`), creating one "subroutine" from each suspend point. After the transformation, each `suspend` will suspend the running program by storing which continuation must be run later and returning control to an external scheduler. The tricky part is to ensure that "resuming" from a subroutine is equivalent to as if the program never yielded at the suspend point in the presence of loops (dealing with conditionals is mostly straightforward). There are two approaches to this: "direct unrolling" and "condition replay". The "direct unrolling" approach basically works by unrolling the remainder of the current iteration of the loop, followed by the whole loop again.

   As an example, direct unrolling transforms the left program to the right program:

```
A;
while (cond1) {
    B;
    if (cond2) {
        suspend;
        C;
    }
    D;
}
E;
```

```
entry:
A;
while (cond1) {
    B;
    if (cond2) {
        frame.next_subr = 1;
        return;
        C;
    }
    D;
}
E;

subroutine 1:
C;
D;
while (cond1) {
    B;
    if (cond2) {
        frame.next_subr = 1;
        return;
        C;
    }
    D;
}
E;
```

`C` and `D` post-dominate the suspend statement, and thus must be executed to finish the loop iteration that was interrupted by the suspend.

On the other hand, "condition replay" works by copying the (possibly nested) set of loops / conditionals leading up to a `suspend`, actually executing branches / loops leading up to suspend point, and using a new `SkipOnReplay` IR instruction to skip over the *first* invocation of instructions that come prior to `suspend`. The main goal of this is to reduce code bloat that can be caused by unrolling from within highly nested loops. An example can be seen in Figure 19 from [1].

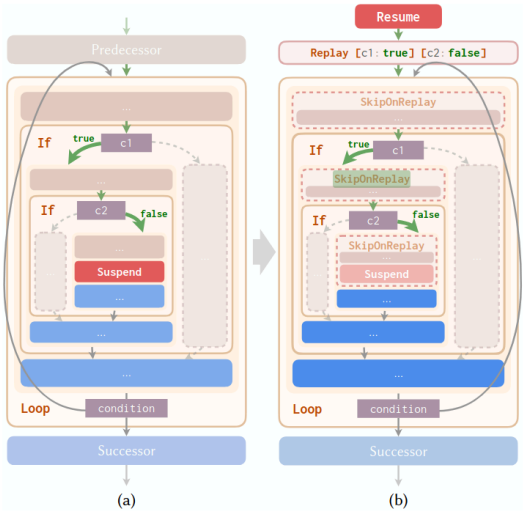


Fig. 19. Control flow reconstruction with condition value replay. To resume the program logic at the suspension point (a), we duplicate the whole related nodes and replay the condition values (b) that have led the control flow (green arrows) to the suspension point. Preceding nodes to the original suspension point are wrapped into a special `SkipOnReplay` node at the first loop entrance after the resumption.

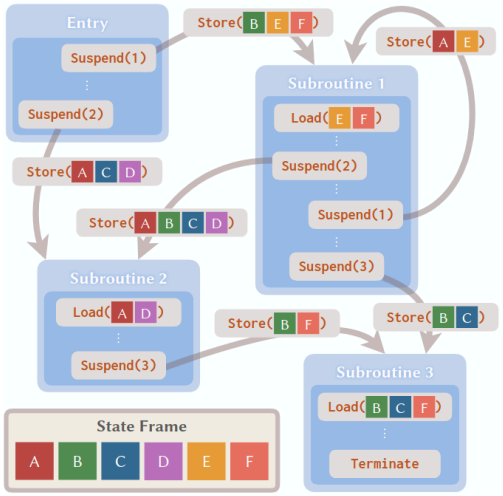


Fig. 6. An example coroutine transition graph. Each node in the graph represents a continuation subroutine. The suspension-resumption relations between them are modeled as directed edges with field-wise usage information of the state frame. The states to load on subroutine resumption and to store on suspension are also recorded in the graph.

2. **Determining the coroutine frame**: once the program has been split into "coroutine scopes", the next step is to figure out which values need to be saved and restored within the "coroutine frame"; this allows live state to be passed between different subroutines. This involves doing dataflow analysis to determine which values a subroutine kills (overwrites), which ones it potentially modifies, and which ones it requires that haven't been overwritten by a previous instruction in the subroutine. With this information, a graph between subroutines with edges containing information about which values need to be kept alive across the transition between two subroutines can be created, and the coroutine frame can be constructed to hold this information. The IR is augmented with appropriate save / load instructions to and from coroutine frame. A visualization of the data dependencies between different coroutine scopes is shown in Figure 6 from [1].

My goal for the project was to formalize a basic model of the IR that LuisaCompute uses (being primarily interested in control flow of the program) as well the operational semantics of running a program with and without coroutines, then implement some of these transforms and prove that they preserve the correctness of the program.

# Implementation Details

## Program Representation

Statements are a representation of the LuisaCompute XIR which directly embed the fact that it only has *structured* control flow. In particular, there are only two types of control flow that we need to worry about: conditionals and loops. Constructs like `break`, `switch`, and `return` can all be mapped down to just conditionals and loops. Since the control flow is *structured*, we can adopt a tree-like structure that makes proofs go through more easily, rather than directly trying to prove things about a linear IR. Specifically, statements like `loop` directly contain the statements making up their body, since it is not possible for an arbitrary jump to enter the loop body. The lack of anything resembling a `goto` means representing the CFG as an actual graph is not required.

Everything except control flow statements is shallowly embedded as a `ShallowInstr (id: ℕ)`. These represent basically everything else the real program would be allowed to do: performing arithmetic, modifying memory, doing IO, launching ray tracing operations, etc. The **state** of a program is represented by a trace of `id`s of `ShallowInstr`s that the program executed. In terms of the mapping from "real" program, you can create two `ShallowInstr` with the same `id`, as long as they are guaranteed to "do the same thing" under the same state. This is a bit hand-wavy, but restricting ourselves to this shallow embedding makes proving properties about this representation much more tractable. In particular, it focuses the proof only on the control flow, and any state manipulation that the program does is abstracted away.

However, this structure would make proving anything about the coroutine frame more difficult, so future work would be to more deeply embed statements, expressions, and program state.

```
structure DecidableCond where
  pred: List ℕ → Prop
  [dec : DecidablePred pred]
  comment: Option String

inductive StmtExt : Type where
  | ShallowInstr (id: ℕ)
  | Sequence (head: StmtExt) (tail: StmtExt)
  | If
    (cond: DecidableCond)
    (then_body: StmtExt)
  | Loop
    (cond: DecidableCond)
    (body: StmtExt)
  | Suspend

structure State where
  trace: List ℕ
```

Originally, basic blocks were represented as `List StmtExt`. However, this led to more complications, requiring mutually recursive functions for splitting statements and splitting lists of statements in the splitting algorithm. Proving the correctness of these mutually recursive functions was very difficult, so in the end I restructured it to instead have `StmtExt` include the `Sequence` constructor, which is much more similar to the `WHILE` language we discussed in class.

Extending this to the coroutine program, it is almost entirely the same, but has `Yield` replacing `Suspend` and also encodes the invariant that the number of subroutines is given by `k`.

```
inductive StmtCo : Type where
  | ShallowInstr (id: ℕ)
```

```
  | Sequence (head: StmtCo) (tail: StmtCo)
  | If
    (cond: DecidableCond)
    (then_body: StmtCo)
  | Loop
    (cond: DecidableCond)
    (body: StmtCo)
  | Yield (next: Fin k)
  | Skip

structure ProgramCo where
  main: @StmtCo k
  subroutines: List (@StmtCo k)
  hsubr_count: subroutines.length = k
```

## Operational Semantics

For the operational semantics, I used big-step semantics. Originally, both the straight-line semantics and coroutine semantics were specified using small-step semantics. However, this was troublesome since dealing with reflexive-transitive-closure was difficult within the proof, and the small-step semantics didn't add much since the language is totally deterministic. Directly doing induction instead with big-step semantics proved much simpler. For coroutine semantics, it was necessary to add a "marker" within the inductive predicate which represent whether there was a yield that occurred while big-stepping some statement. Without this, the yield instruction would big-step to the result of running the pointed-to subroutine, but then the instructions after the yield would also be big-stepped as part of the `Sequence` or `LoopContinue` constructor. The marker is propagated up the chain of `CoroutineSteps` from `Yield`, and `SequenceEarlyYield` and `LoopEarlyYield` use this to make sure that they only step the part of the program leading up to the yield and nothing after it.

```
inductive Outcome : Type where
| Yielded
| Completed

inductive CoroutineStep {program: @ProgramCo k} :
  ((@StmtCo k) × State) → State → Outcome → Prop where
| ShallowInstr (id: ℕ)
  (s: State) :
  CoroutineStep (StmtCo.ShallowInstr id, s) (s.update id) Outcome.Completed
| SequenceNormal (A B : StmtCo)
  (a b c: State)
  (B_outcome: Outcome)
  (hA : @CoroutineStep program (A, a) b Outcome.Completed)
  (hB : @CoroutineStep program (B, b) c B_outcome) :
  CoroutineStep (StmtCo.Sequence A B, a) c B_outcome
| SequenceEarlyYield (A B : StmtCo)
  (a b : State)
  (hA : @CoroutineStep program (A, a) b Outcome.Yielded) :
  CoroutineStep (StmtCo.Sequence A B, a) b Outcome.Yielded
 -- <rest omitted for brevity>
```

Listing 1: Definition of `CoroutineStep`, highlighting the `Outcome` marker which represents early yield that interrupts normal control flow

The first attempt to operationalize the coroutine semantics didn't have the `Outcome` marker, and just had the `Yield` constructor directly big-stepping the appropriate subroutine.

```
| Yield (next: Fin k)
  (s t: State)
  (hsubr : @CoroutineStep program (program.subroutines[next]'(by simp
[program.hsubr_count]), s) t) :
  CoroutineStep (StmtCo.Yield next, s) t
```

However, this doesn't really work on its own, since you can consider a program like

```
A;
suspend;
B;
```

The result of splitting the program might look like

```
main:
  SI 0;
  Yield 0;
  SI 1

subr_0:
  SI 1;
  Skip
```

The main issue lies in the fact that the yield extends the big-step to the result of the subroutine, so the state after yield is `[0, 1]`. But, the `Sequence` containing the yield would still big-steps the other arm, and causes the state to be updated again to `[0, 1, 1]`. The addition of `Outcome` fixes this by setting `Outcome.Yielded`, and thus the only way to construct a coroutine step for a sequence with `(Yield 0; SI 1)` in sequence is through the `SequenceEarlyYield` constructor, which correctly skips the other arm.

## Algorithm

The splitting algorithm maps from a `StmtExt` to a `ProgramCo`. It is a relatively straightforward recursive algorithm, since most statements can be directly mapped from `StmtExt` to `StmtCo`. Essentially, at every call to `splitStmt`, there is a `cont: StmtCo` argument which represents the current continuation of the program and `subr_index: ℕ` which represents the current size of the subroutine list. `Skip` is used as the continuation of the full program, which results in every subroutine ending with `Skip` no-op. When a `Suspend` statement is encountered, it just appends `[cont]` onto the list of subroutines and creates a `Yield subr_index`. Other cases just maintain the invariant that `cont` should represent the continuation of the program for recursive calls to `splitStmt`.

There are two complications:

1. We need to establish the invariant that the number of subroutines matches `k`, and that `subr_index` is less than `k`. To do this, a proof that the number of created subroutines equals the number of `Suspend` statements and that `subr_index` plus the number of created subroutines `≤ k` is threaded through the recursive calls.
2. In order to do direct unrolling, the loop being processed must itself be part of the continuation. To do this, the body is split using a dummy `Skip` continuation. The resulting `StmtCo` is wrapped in `StmtCo.Loop` then prepended to the real continuation for a second recursive call to `splitStmt`.

## Testing

To test the correctness of the algorithm, I made some basic test programs with some syntax sugar (already shown above) to write it more nicely as well as a pretty printer to make the program structure more clean (both are separated into `namespace tests`).

Then, I wrote a basic recursive "interpreter" which would "run" both coroutine programs and straight line programs, returning the final state as well as a `StraightLineStep` or `CoroutineStep` which witnesses that the operational semantics are respected. The requirement that the condition is decidable (encoded by `DecidableCond`) arises here, and is not used in the proof. I thought that this part was interesting, since the requirement to return a value of the inductive predicate type meant that while implementing, I got instant feedback from Lean's type-checker about whether the step the interpreter "performed" was actually valid. This makes writing the interpreter incorrectly difficult, which was a pleasant surprise.

One slightly tricky aspect was satisfying the termination checker. The actual correctness proof requires a proof that the program will halt with some specified output on some given input, which sidesteps this problem, but the whole point of the "interpreter" is that it will compute the output, and it is certainly possible to make a non-terminating program. To deal with this, there is a `fuel` parameter that Lean is able to use to prove termination in the two cases (`LoopContinue` and `Yield`) which doesn't create a structurally "smaller" input to the recursive call. Then, when running a test case, I just picked a large number to be the fuel. I tried using `partial def` as well, so that the termination checker would not complain, but this was an issue due to `partial def` requiring the type to be inhabited. The subtype is not inhabited automatically, since it contains a predicate that Lean cannot know a priori to hold for a default element. I considered adding a typeclass instance for it, but a `fuel` parameter seems more principled in that it will not crash the Lean server if non-termination actually does occur.

Finally, as a bit of fun, I had Claude generate a factorial program to run. It gets the same answer (5! = 120) for both straight-line and coroutine interpreters, which is good. Pretty much all of the computational power of the "language" comes from the conditionals, since the `ShallowInstr` cannot inspect the state at all. For example, to implement the loop `while (i < k)`, where `k` is the input to factorial (represented in unary by `k` zeroes in the input tape), the body contains `ShallowInstr 1`, and the condition is just checking if the number of ones in the state is less than the number of zeroes. Multiplication is done in a similar manner. I believe that the model of computation here is technically Turing-complete.

## Proof Structure

The goal of the primary proof is as follows: "for all straight-line programs that halt, the final state is equal to the the state of the split program run using coroutine semantics".

The primary meat of the proof is in the `splitStmtSimulation` lemma, whose type signature is replicated here

```
lemma splitStmtSimulation
    (program : @ProgramCo k)
    (stmt : StmtExt)
    (cont : @StmtCo k)
    (subr_index : ℕ)
    (hbound : subr_index + countSuspendsStmt stmt ≤ k)

    (s t u : State)
    (cfg : StmtExt × State)
    (hcfg : cfg = (stmt, s))
    (hrun : StraightLineStep cfg t)

    (stmt_co : @StmtCo k)
    (subrs : List (@StmtCo k))
    (new_subr_index : ℕ)
```

```
    (hindex : new_subr_index = subr_index + countSuspendsStmt stmt)
    (hlen : subrs.length = countSuspendsStmt stmt)
    (hsplit : splitStmt stmt cont subr_index hbound = ⟨(stmt_co, subrs,
new_subr_index), ⟨hindex, hlen⟩⟩)

    (hwell_formed : ∀ i (hi : i < subrs.length),
      program.subroutines[subr_index + i]'(by rw [program.hsubr_count]; rw [hlen] at
hi; omega) = subrs[i])

    (cont_outcome : Outcome)
    (hcont : @CoroutineStep k program (cont, t) u cont_outcome) :
    ∃ outcome,
    (outcome = Outcome.Completed ∧ @CoroutineStep k program (stmt_co, s) t outcome) ∨
    (outcome = Outcome.Yielded ∧ @CoroutineStep k program (stmt_co, s) u outcome)
```

We are given that `(stmt, s)` straight-line-steps to `t`. Essentially, the proof boils down to just two cases:

1. The big-step of `stmt` didn't contain a suspend. This corresponds to `outcome = Outcome.Completed`, and we must prove that `(stmt_co, s)` coroutine-steps to `t`
2. The big-step of `stmt` did contain a suspend. This corresponds to `outcome = Outcome.Yielded`. Given a proof that the continuation of `stmt` coroutine-steps `(cont, t)` to `u`, then we must prove that `(stmt_co, s)` coroutine-steps to `u`. In practice, `u` always ends up being the final state of the program.

The existential and disjunction as the output type directly encodes these two cases. The overall proof uses induction on `hrun`. Some common patterns in the proof:

- The `induction` tactic doesn't do well when the induction variable contains non-variable terms in its type, which means that it is needed to generalize `(stmt, s)` into a `cfg` and have `hcfg` hypothesis. However dealing with this in the body of the proof is annoying, so using the `subst` tactic to undo this generalization helps prune the tactic state to be more manageable. Similarly, lean sometimes fails to rewrite dependent types without explicit guidance and usages of `subst`. I think there might have been an opportunity to simplify by writing a tactic that would substitute a whole tuple (nested product) of equalities, but I'm not sure whether this is useful or more a symptom of me writing unidiomatic Lean.
- In cases where there was no recursive call to `splitStmt` within the match arm (e.g. `ShallowInstr`), lean can compute through the `splitStmt` in `hsplit` after using `rw` tactic. In other cases, it cannot, and the type of the term is a match statement which is effectively just the body of the function. In this case, I needed to use the `split` tactic to destructure the match statement, even though it only has one arm. This would inject the results of a recursive call to `splitStmt` into the tactic state, and a hypothesis that those variables come from the recursive `splitStmt` call. Then, this could be used for applying the inductive hypothesis to get that the head / tail (for `Sequence`) or the body of a conditional / loop either yielded or completed.
- The well-formedness of the program, as given by `hwell_formed`, guarantees that the passed in `program` matches what was actually generated by `splitStmt`. However, it actually doesn't require that the privileged entry subroutine matches the straight-line program. This is handled by the main theorem definition, which operates on the full program.

```
theorem splitPreservesSemantics
(program : ProgramExt)
(initial_state: List ℕ)
(final_state: List ℕ)
(hrun : StraightLineStep (program.stmt, ⟨initial_state⟩) ⟨final_state⟩)
(split_program : @ProgramCo (countSuspendsStmt program.stmt))
```

```
(hsplit_program : split_program = split program):
∃outcome,
@CoroutineStep
  (countSuspendsStmt program.stmt)
  split_program
  (split_program.main, ⟨initial_state⟩) ⟨final_state⟩
  outcome
```

In this case, it has a precondition that the program actually terminates (`hrun`), and that the entry subroutine matches what `split` (which uses `splitStmt`) would produce. The definition of this theorem presumes that the program terminates, and makes no guarantees about a non-terminating program having the same semantics under the transform. One could imagine an oracular `split` function which can determine if a straight line program will loop forever on some input and output a coroutine program which terminates under the same initial condition. Big-step semantics are not well-suited for reasoning about non-terminating programs, so adding this would be pretty nontrivial (effectively the same proof in reverse).

## Summary

Overall, this project was a success. The main theorem about the splitting transform successfully preserving the semantics of the program was proven. The clearest direction for future work is just implementing condition replay and the coroutine-frame dataflow analysis.

The `Basic.lean` file contains roughly 1000 lines of Lean, with around 100 to define the program representation and operational semantics, 100 for implementing the transform, 300 for tests and pretty printing, and 500 lines for the proof. The time spent on the proof was much longer than all the other parts put together, even though the algorithm is "obviously" correct, from my perspective. This was a fun exercise to see how even proving simple program transformation in a highly simplified environment can be rather involved.

## References


[1] S. Zheng, X. Chen, Z. Shi, L.-Q. Yan, and K. Xu, "GPU Coroutines for Flexible Splitting and Scheduling of Rendering Tasks," *ACM Trans. Graph.*, vol. 43, no. 6, Nov. 2024, doi: 10.1145/3687766.